Load Balancer per calcolo distribuito su GPU

Enrico Covili

25 gennaio 2026

Indice

1 Introduzione **2**

2 Analisi dei Requisiti **3**
2.1 Requisiti Funzionali 3
2.2 Requisiti Non Funzionali 3
2.3 Necessità di un Ambiente di Simulazione 3

3 Stile Architetturale **4**
3.1 L'Orchestratore e i Worker 4
3.2 Vantaggi dell'approccio Orchestrato 4
3.3 Linguaggio 4

4 Modellazione UML **5**
4.1 Use-Case Diagram 5
4.2 Class Diagram 6
4.3 Sequence Diagram: Inserimento Nuovo Job 8
4.4 Activity Diagram 9
4.5 State Diagram: Stato di un Job 10

5 Progettazione Software e Design Pattern **11**
5.1 Strategy Pattern 11
5.2 Singleton Pattern 13
5.3 Factory Method Pattern 14

6 Conclusioni e Sviluppi Futuri **15**
6.1 Sintesi dei Risultati 15
6.2 Sviluppi Futuri 15

1 Introduzione

In un'era dominata dal calcolo dedicato all'intelligenza artificiale e agli algoritmi di Machine Learning, le GPU sono diventate una risorsa preziosa e spesso mal utilizzata. Uno dei principali problemi che si riscontra nei cluster di calcolo è la distribuzione inefficiente della mole di lavoro tra singole unità.

Questa tesi illustra la progettazione di un software in grado di agire come intermediario tra le richieste di elaborazione e le risorse fisiche. Attraverso l'utilizzo di uno scheduler dedicato il prototipo sviluppato mira a ottimizzare l'utilizzo delle GPU, garantendo velocità e riduzione della latenza senza sacrificare la stabilità del sistema.

2 Analisi dei Requisiti

2.1 Requisiti Funzionali

- **Monitoraggio delle Risorse in Tempo Reale:** Il sistema deve interrogare periodicamente lo stato di ogni GPU nel cluster, recuperando metriche critiche quali l'utilizzo della memoria video (VRAM), la percentuale di utilizzo dei core (GPU Load) e la temperatura.

- **Gestione delle Richieste (Job Dispatching):** Il sistema si dovrà occupare di ricevere le richieste da un client e inserirle in una coda, ordinata secondo una certa proprità. Successivamente questi "job" saranno assegnati alle singole GPU secondo un preciso algoritmo di scheduling.

- **Gestione dei Fallimenti:** In caso di crash di un processo o disconnessione improvvisa di una risorsa hardware, il sistema deve rilevare l'evento e ri-schedulare il job interrotto su un'altra unità disponibile, garantendo la continuità del servizio.

2.2 Requisiti Non Funzionali

- **Bassa Latenza (Overhead Minimo):** Il tempo impiegato dal load balancer per decidere su quale GPU allocare un task deve essere trascurabile rispetto al tempo di esecuzione del task stesso. L'overhead introdotto dall'orchestratore non deve degradare le performance del sistema.

- **Scalabilità Orizzontale:** L'architettura deve supportare l'aggiunta dinamica di nuove unità di calcolo al cluster senza richiedere il riavvio dell'intero sistema o modifiche al codice sorgente.

- **Robustezza e Availability:** Il sistema deve garantire un uptime elevato. Picchi di carico o eventuali errori non devono poter degradare lo stato del sistema in maniera significativa.

- **Sicurezza:** Essenziale assicurare che la comunicazione tra nodi e orchestratore sia blindata e sicura, in modo che i dati scambiati non siano visualizzabili all'esterno.

- **Usabilità:** Il software deve essere semplice anche nel deploy. Una soluzione troppo complessa causerebbe problemi di rilascio e manutenzione futura.

2.3 Necessità di un Ambiente di Simulazione

Estrema importanza, specialmente se si lavora con hardware complesso, viene data alla validazione del software. Dato l'elevato costo delle risorse, non è sostenibile condurre le fasi iniziali di testing e debugging direttamente sulla futura infrastruttura di produzione.

L'accesso concorrente alle risorse fisiche durante la fase di sviluppo potrebbe causare blocchi hardware, surriscaldamenti o inconsistenze difficili da replicare. Pertanto, è stato definito come requisito vincolante lo sviluppo di un ambiente di simulazione.

Questo modulo software deve emulare il comportamento delle GPU, generando metriche adeguate (es. simulando un picco di memoria o un ritardo nella risposta) e

permettendo di stressare l'algoritmo di bilanciamento in scenari limite. Solo dopo aver convalidato le performance e la robustezza del sistema in ambiente simulato sarà possibile il deploy su hardware reale.

3 Stile Architetturale

Sebbene il sistema possa essere distribuito come una singola unità logica, la sua progettazione interna riflette i principi fondamentali dell'architettura a **Microservizi con Orchestrazione**.

3.1 L'Orchestratore e i Worker

Nel paradigma a microservizi, un Orchestrator è responsabile della gestione del ciclo di vita dei processi e della distribuzione del traffico verso servizi specializzati. Possiamo quindi fare le seguenti analogie applicate al nostro contesto:

- **Load Balancer come Orchestrator:** Un elemento centrale si occuperà di validare e distribuire le richieste mentre controlla lo stato del sistema .
- **GPU come Microservizi:** Ogni unità di elaborazione (reale o simulata) ha il ruolo di microservizio singolo, che non conosce lo stato del resto del sistema. Questo ci permette di raggiungere un buon disaccoppiamento non sono a livello di codice ma anche nella logica progettuale.

3.2 Vantaggi dell'approccio Orchestrato

L'utilizzo di questo stile architetturale porta numerosi vantaggi:

1. **Disaccoppiamento:** Il client comunica esclusivamente con l'orchestratore. Non ha bisogno di sapere alcuna informazione sul sistema (tipo di GPU, numero di nodi attivi...)
2. **Scalabilità Orizzontale:** Poiché i le GPU sono viste come servizi è immediato inserire nuovi elementi nell'infrastruttura senza modifiche o riavvii del sistema.
3. **Resilienza:** Con questo approccio, se una GPU va in crash, l'Orchestrator registra il guasto e può riassegnare il task su un altro nodo attivo.

Questa struttura prepara il sistema a future evoluzioni verso ambienti distribuiti su cloud, dove l'Orchestrator e i Worker potrebbero risiedere su macchine fisiche differenti comunicando via rete.

3.3 Linguaggio

Il linguaggio utilizzato è Java. La scelta è stata dettata dalla ampia scelta di librerie disponibili per lo scopo del software. Non abbiamo bisogno di usare linguaggi più perfomativi come C/C++ in quanto l'overhead introdotto da Java si rivela pienamente accettabile.

4 Modellazione UML

Vengono qui rappresentati i diagrammi principali che descrivono sia la struttura statica che il comportamento dinamico del software.

4.1 Use-Case Diagram

Il diagramma dei casi d'uso illustra l'interazione tra gli attori principali e le funzionalità offerte dal sistema. Gli attori sono:

- **Client/Utente:** L'entità che richiede l'esecuzione di un task.
- **Amministratore:** Responsabile della configurazione del cluster e del monitoraggio.
- **Sistema (Load Balancer)**
- **GPU (Reale o Simulata):** L'attore che esegue il calcolo.

I casi d'uso principali includono l'invio di un nuovo Job, la richiesta dello stato di avanzamento, l'aggiunta dinamica di nodi al cluster e la simulazione di guasti hardware.

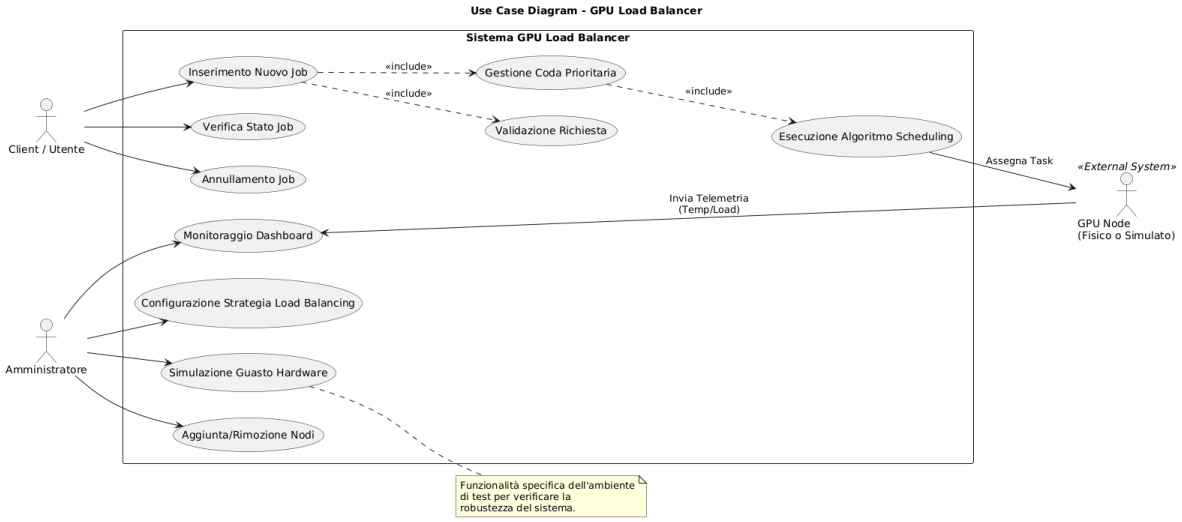

Figura 1: Use Case Diagram

4.2 Class Diagram

In questa sezione viene presentato il modello strutturale del sistema, cercando di inserire nel diagramma le classi principali che lo compongono.

Le classi principali:

- **LoadBalancerOrchestrator:** È la classe "core" del sistema. Funge da controllore centrale, gestendo la coda dei `Job` in ingresso e mantenendo la pool delle risorse `GPU` disponibili. Contiene la logica del loop di scheduling.
- **Client:** L'entità esterna che sottomette richieste di elaborazione al sistema.
- **Administrator:** L'attore responsabile della configurazione dei parametri del bilanciatore e del monitoraggio dello stato di salute del cluster.
- **Job:** Rappresenta l'unità di lavoro atomica. Contiene i metadati necessari per lo scheduling (ID, priorità, stato) e il payload dati vero e proprio.
- **GPU:** Classe astratta che definisce l'interfaccia generica di una unità di elaborazione. Permette al sistema di trattare uniformemente hardware reale e simulato (Polimorfismo).
- **RealGPU** e **SimulatedGPU:** Implementazioni concrete della classe astratta `GPU`. La prima si interfaccia con i driver hardware fisici (es. CUDA), la seconda implementa una logica "mock" per ambienti di test, potendo simulare guasti o ritardi.
- **GPUResource:** Una classe dati che incapsula le metriche di una GPU (VRAM utilizzata, carico percentuale, temperatura) in un determinato istante.

Il diagramma evidenzia come il `LoadBalancerOrchestrator` agisca da aggregatore, gestendo una relazione uno-a-molti sia con i `Job` in coda che con il pool di oggetti `GPU`.

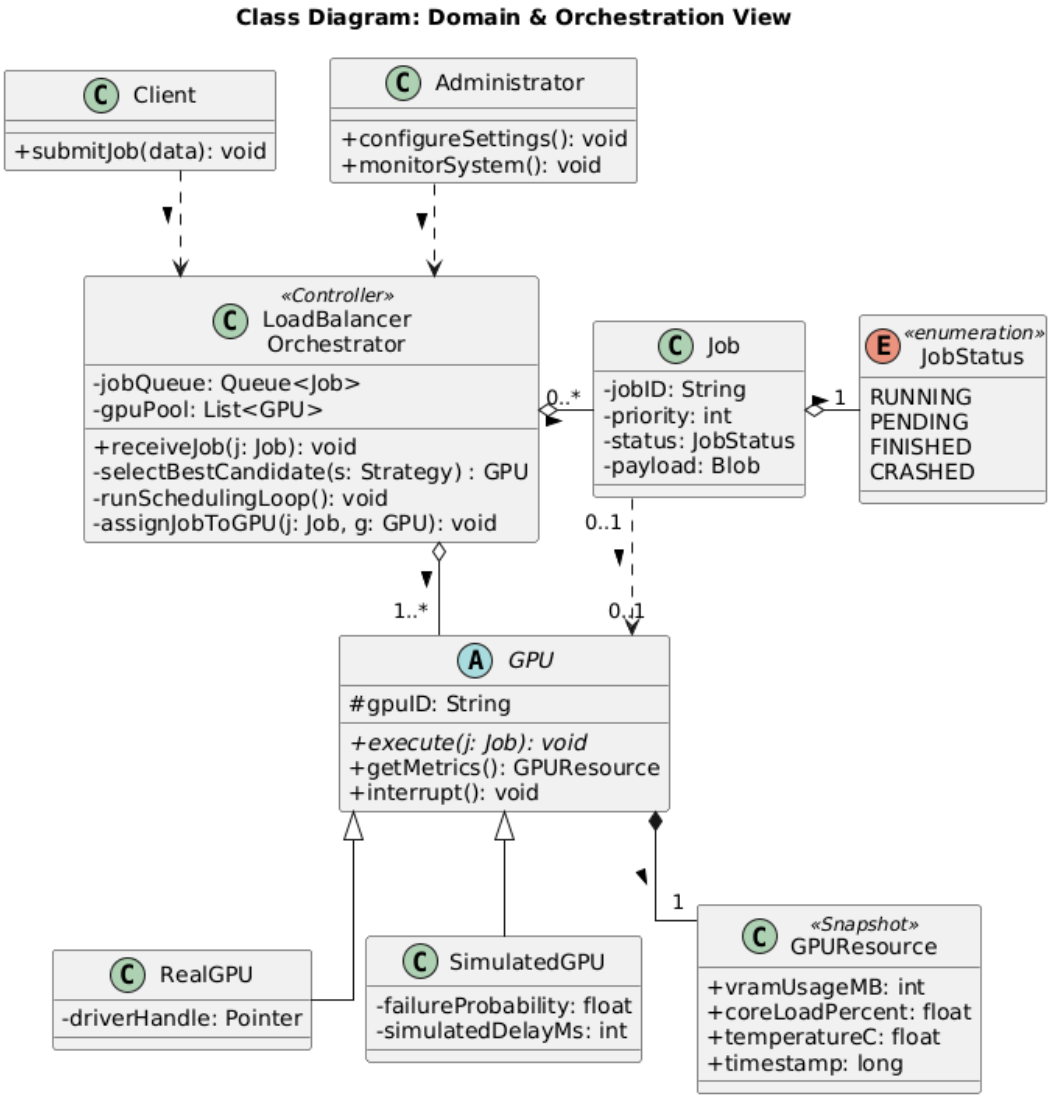

Figura 2: Class Diagram Generale

4.3 Sequence Diagram: Inserimento Nuovo Job

Il Sequence Diagram modella lo scenario di sottomissione di un nuovo task di lavoro:

1. Il **Client** invia una richiesta contenente i dati del task.
2. Il **Controller** riceve la richiesta e istanzia un oggetto `Job`.
3. Il Job viene inserito nella coda gestita dallo Scheduler.
4. Lo **Scheduler** notifica l'avvenuta presa in carico al Client (risposta asincrona).
5. In un thread separato, lo Scheduler assegna la miglior GPU disponibile secondo l'algoritmo di scheduling.

Questo diagramma evidenzia la natura asincrona e non bloccante dell'architettura scelta.

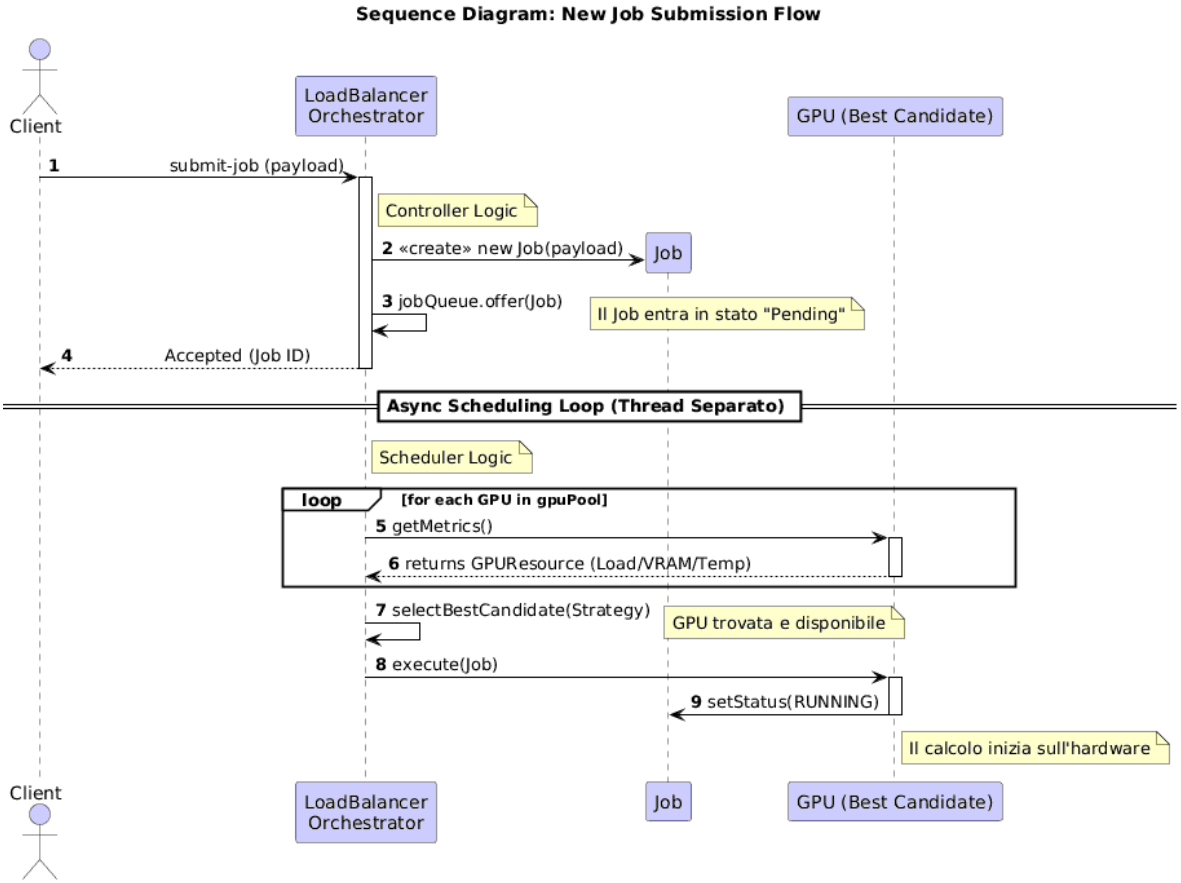

Figura 3: Sequence Diagram

4.4 Activity Diagram

L'activity diagram è assolutamente analogo al sequence diagram. Si vuole mettere in evidenza la natura asincrona del sistema e il flusso di operazioni che vengono svolte durante la sottomissione di una task da parte del client.

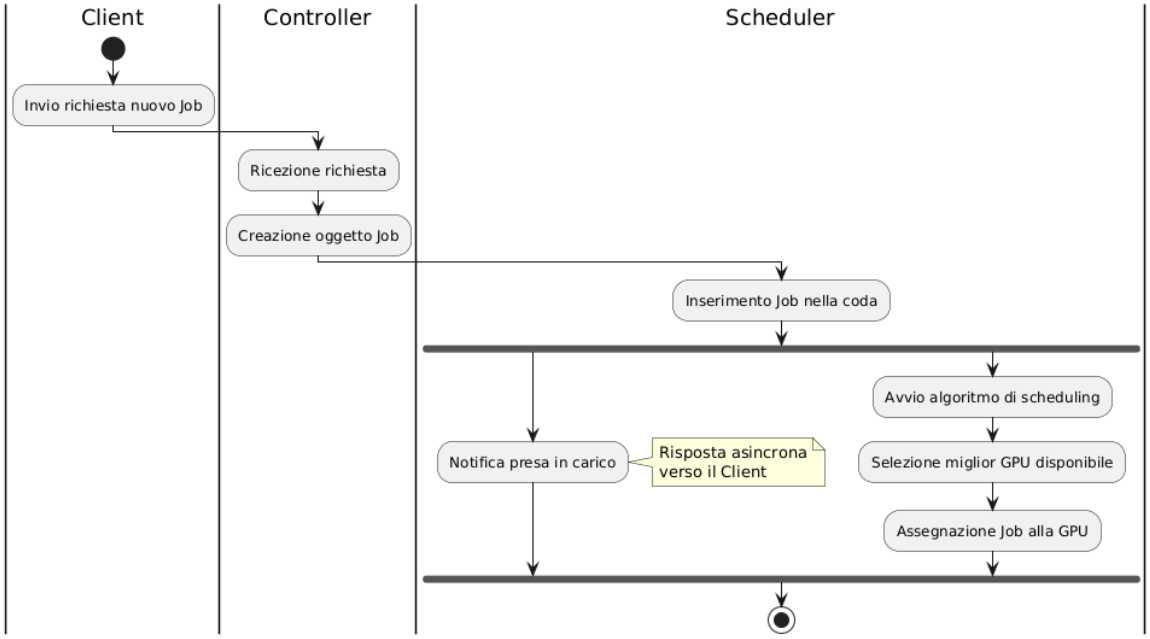

Figura 4: Activity Diagram

4.5 State Diagram: Stato di un Job

Segue il diagramma a stati finiti realizzato per descrivere lo stato di un singolo carico di lavoro (`Job`).

Gli stati sono:

- **PENDING:** Stato iniziale. Il Job è stato inserito in una coda e sta aspettando di essere assegnato a una GPU dallo scheduler.
- **RUNNING:** Lo scheduler ha assegnato il Job a una GPU e l'elaborazione è in corso sull'hardware (reale o simulato).
- **FINISHED:** L'elaborazione è terminata con successo e i risultati sono pronti per essere recuperati dal client.
- **CRASHED:** L'elaborazione è stata interrotta a causa di un'eccezione non gestita, un timeout o un guasto hardware simulato.

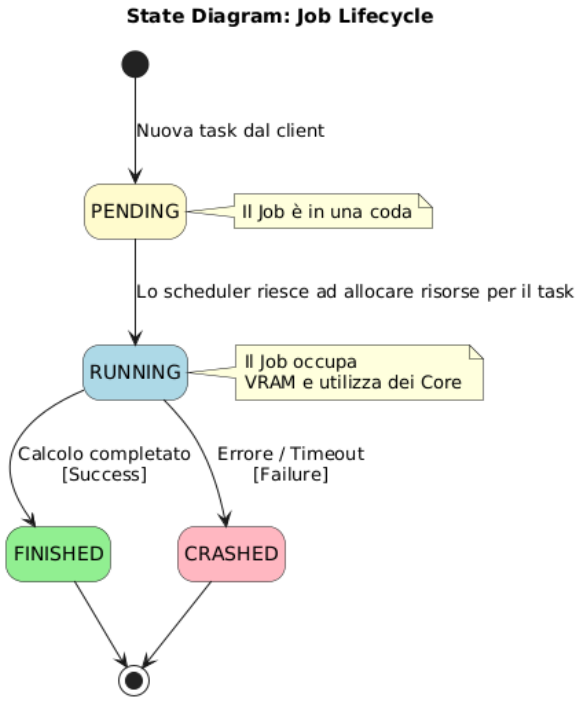

Figura 5: State Diagram per un Job

5 Progettazione Software e Design Pattern

La progettazione del sistema è stata affiancata dall'utilizzo di design pattern scelti per garantire modularità, estensibilità e manutenibilità del codice. In particolare, l'architettura risponde alla necessità di disaccoppiare la logica di business (lo scheduling) dall'implementazione fisica delle risorse (l'hardware GPU).

Di seguito vengono analizzati i pattern principali selezionati.

5.1 Strategy Pattern

Il cuore del sistema risiede nella decisione di quale nodo debba eseguire un determinato task. Poiché non esiste un algoritmo di bilanciamento universalmente ottimo per ogni scenario, è stato adottato lo **Strategy**.

Questo pattern comportamentale permette di definire una famiglia di algoritmi (le strategie di scheduling), incapsularli in classi separate e renderli intercambiabili a runtime. Nel contesto del Class Diagram (Fig. 2), il `LoadBalancerOrchestrator` non implementa direttamente la logica di scelta, ma delega questa responsabilità a un'interfaccia `ISchedulingStrategy`.

Ciò consente di passare dinamicamente da una strategia all'altra, senza modificare in alcun modo il codice attuale.

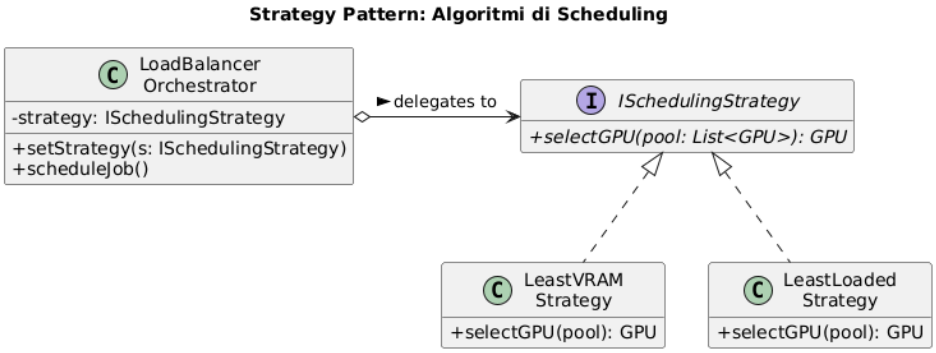

Figura 6: Class Diagram ridotto per mostrare lo Strategy Pattern

```java
public interface ISchedulingStrategy {
    GPU selectBestNode(List<GPU> gpuPool);
}

public class LeastLoaded implements ISchedulingStrategy {
    @Override
    public GPU selectBestNode(List<GPU> gpuPool) {
        ...
    }
}

public class LeastVRAM implements ISchedulingStrategy {
    @Override
    public GPU selectBestNode(List<GPU> gpuPool) {
        ...
    }
}

public class LoadBalancerOrchestrator {
    private ISchedulingStrategy strategy;
    private List<GPU> gpuPool;

    public LoadBalancerOrchestrator(ISchedulingStrategy strategy
       ) {
        this.strategy = strategy;
    }

    public void setStrategy(ISchedulingStrategy strategy) {
        this.strategy = strategy;
    }

    public void dispatchJob(Job job) {
        GPU target = this.strategy.selectBestNode(this.gpuPool);
        if (target != null) {
            target.execute(job);
        }
    }
}
```

Figura 7: Implementazione Java del pattern Strategy.

5.2 Singleton Pattern

La classe `LoadBalancerOrchestrator` agisce come punto centrale di controllo per l'intero sistema. Per evitare conflitti nell'assegnazione delle risorse e race conditions sulla coda dei Job, è essenziale che esista una sola istanza di questa classe durante l'esecuzione.

È stato quindi applicato il **Singleton**. Questo garantisce che l'accesso alla `JobQueue` e al `GPUPool` sia serializzato attraverso un unico punto di accesso globale, prevenendo stati inconsistenti (es. due controller che assegnano lo stesso Job a due GPU diverse).

```java
public class LoadBalancerOrchestrator {
    private static LoadBalancerOrchestrator instance;

    private LoadBalancerOrchestrator() {}

    public static synchronized LoadBalancerOrchestrator
       getInstance() {
        if (instance == null) {
            instance = new LoadBalancerOrchestrator();
        }
        return instance;
    }

    public void addJobToQueue(Job job) {
        // Logica di accodamento centralizzata
    }
}
```

Figura 8: Implementazione del Singleton Pattern (Lazy Initialization thread-safe).

5.3 Factory Method Pattern

Uno dei requisiti non funzionali è la capacità di operare sia su hardware reale che in ambiente simulato. Per risolvere il problema della creazione degli oggetti `GPU` senza legare il codice alle classi concrete, è stato utilizzato il **Factory Method**.

Una classe dedicata, la `GPUFactory`, è responsabile dell'istanziazione degli oggetti.

Grazie a questo pattern e al polimorfismo della classe astratta `GPU`, il resto del sistema (incluso l'Orchestrator) rimane distaccato dalla natura dell'hardware.

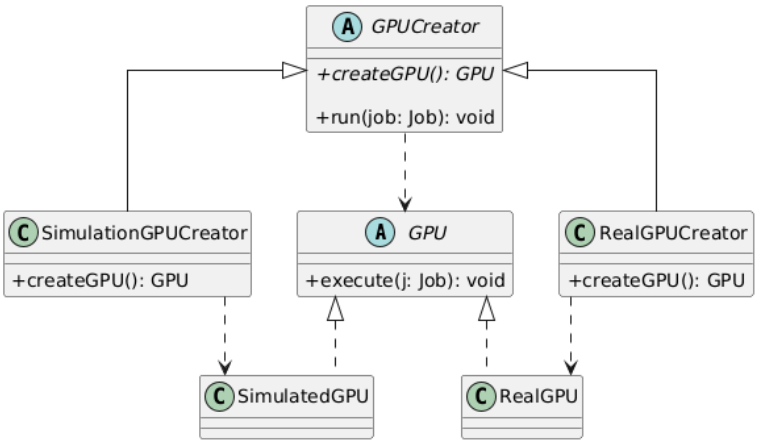

Figura 9: Class Diagram ridotto per mostrare il Factory Method Pattern

```
public abstract class GPUCreator {
    public abstract GPU createGPU();

    public void run(Job job) {
        GPU gpu = createGPU();
        gpu.execute(job);
    }
}

public class SimulationGPUCreator extends GPUCreator {
    @Override
    public GPU createGPU() {
        return new SimulatedGPU();
    }
}

public class RealGPUCreator extends GPUCreator {
    @Override
    public GPU createGPU() {
        return new RealGPU();
    }
}
```

Figura 10: Implementazione del Factory Method per la creazione delle GPU.

6 Conclusioni e Sviluppi Futuri

Il lavoro svolto in questa tesi ha portato alla progettazione di un sistema di Load Balancing per risorse GPU, affrontando criticità tipiche degli ambienti di calcolo ad alte prestazioni: la gestione della concorrenza, l'ottimizzazione delle risorse e il raggiungimento della massima efficienza possibile nel contesto.

6.1 Sintesi dei Risultati

L'architettura proposta ha dimostrato di soddisfare i requisiti iniziali grazie a un approccio fortemente modulare. In particolare si evidenzia come l'uso di design pattern abbia migliorato il software:

- L'uso dello **Strategy Pattern** ha reso il sistema flessibile, permettendo di adattare l'algoritmo di scheduling al tipo di carico di lavoro durante il runtime.
- L'implementazione del **Factory Method** e l'astrazione delle risorse tramite il concetto di *SimulatedGPU* hanno permesso di sviluppare e testare la logica di orchestrazione in un ambiente sicuro, riducendo i costi e tempi legati al testing.

6.2 Sviluppi Futuri

Nonostante il sistema attuale rappresenti una solida base, sono possibili alcune estensioni o migliorie da poter implementare nel caso di un'adozione del software a livello di produzione industriale avanzata:

1. **Scheduling Predittivo con Machine Learning:** In questa versione gli algoritmi di bilanciamento si basano su metriche istantanee (carico attuale o utilizzo di VRAM). Uno sviluppo futuro prevede l'integrazione di un modello di Machine Learning capace di stimare la durata e le risorse necessarie per un Job prima della sua esecuzione, permettendo uno scheduling predittivo anziché statico.
2. **Politiche di Resilienza (Fault Tolerance Avanzata):** Attualmente un Job in stato `CRASHED` termina il suo ciclo di vita. Si prevede l'introduzione di un meccanismo di recupero del Job assegnandolo a una nuova GPU.
3. **Containerizzazione e Kubernetes:** Il software, anche se non direttamente esplicitato, è costruito per gestire processi diretti. Uno sviluppo interessante potrebbe essere quello di integrarlo a sistemi distribuiti (come kubernetes) per avere massima adattabilità a sistemi reali già utilizzati in produzione.
4. **Supporto per Cluster di GPU:** Per ora l'architettura proposta permette di effettuare un bilanciamento del carico con un processo per GPU: una estensione futura potrebbe permettere di vedere più GPU come una unica per l'orchestratore, andando a implementare uno scheduler "a più livelli" (uno dal punto di vista di ripartizione delle task e l'altro a livello di singoli elementi hardware).